

Informe Técnico: Arquitectura y Funcionamiento del Sistema Origen X

Benjamin Cruzado, Enzo Loren, Ignacio Morales, Paolo Paredes

1. Descripción General del Sistema

El sistema Origen X es una plataforma distribuida de alto rendimiento diseñada para la gestión integral y reserva de habitaciones de hotel. Su propósito fundamental es ofrecer una infraestructura resiliente que permita a los usuarios interactuar con inventarios en tiempo real bajo condiciones de alta demanda.

Público Objetivo

La plataforma ha sido estructurada para satisfacer dos perfiles de usuario críticos:

- **Clientes Finales:** Usuarios que requieren una interfaz ágil y segura para la búsqueda y confirmación de alojamientos.
- **Operadores Hoteleros:** Administradores que necesitan un backend robusto, escalable y capaz de centralizar operaciones complejas.

Problemas que resuelve

Desde una perspectiva de ingeniería, Origen X mitiga retos inherentes a los sistemas de reservas a gran escala:

1. **Centralización de Inventario:** Actúa como la "fuente de verdad" única, unificando catálogos dispersos en una estructura de datos coherente.
2. **Alta Concurrencia:** Mediante el aislamiento de servicios, el sistema maneja múltiples solicitudes simultáneas de búsqueda y reserva sin comprometer el rendimiento global.
3. **Integridad Distribuida:** Implementa mecanismos de comunicación que aseguran la consistencia de los datos en un entorno donde intervienen múltiples unidades lógicas independientes.

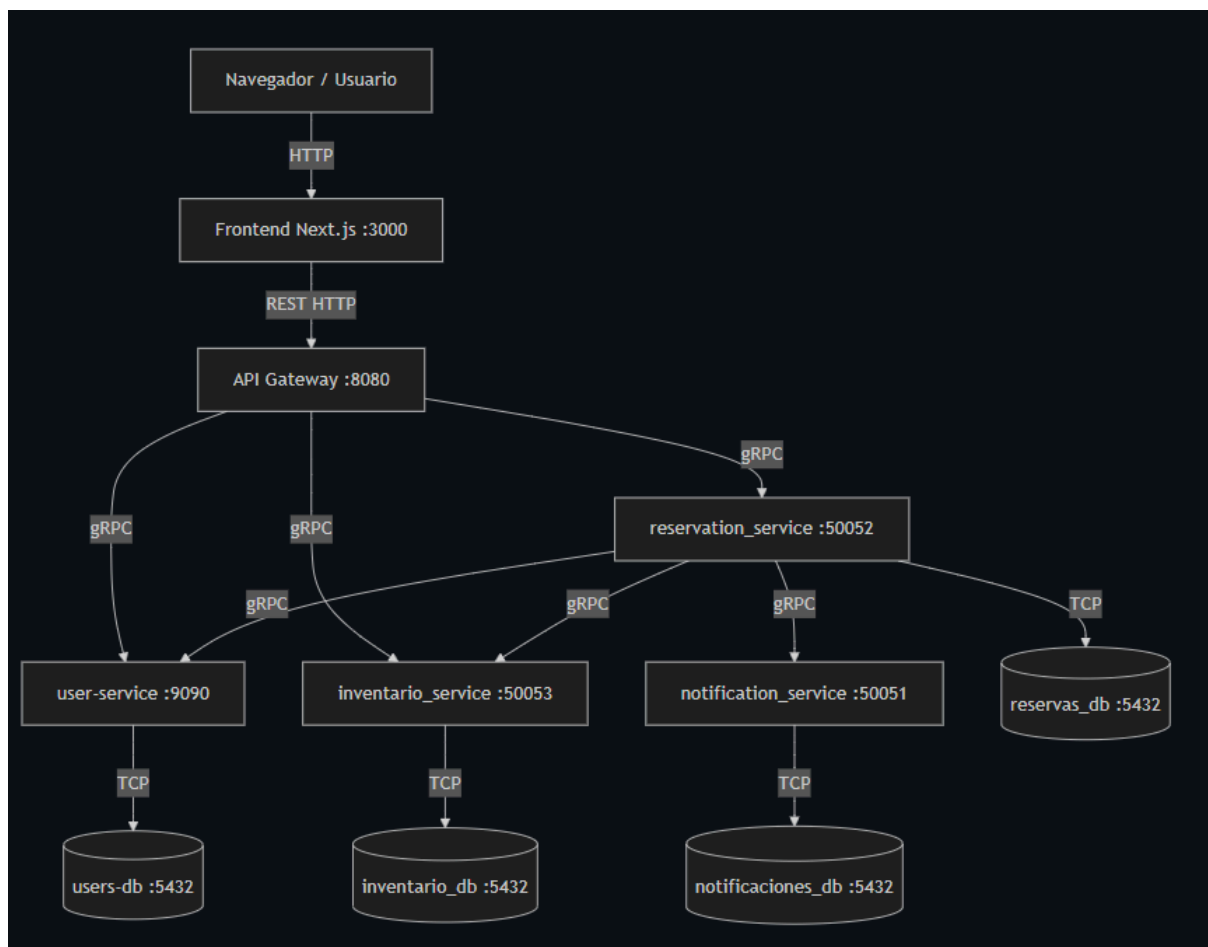
2. Arquitectura del Sistema y Diagrama de Componentes

El sistema adopta una arquitectura orientada a microservicios, utilizando un patrón de Backend For Frontend (BFF) a través de su API Gateway para optimizar la comunicación entre el cliente y los servicios internos.

Componentes y Responsabilidades

- **Frontend (Next.js):** Aplicación cliente que consume la API expuesta por el Gateway. Se comunica vía HTTP/REST en el puerto :3000.
- **API Gateway (FastAPI):** Centraliza el tráfico externo en el puerto :8080. Actúa como un **cliente de Protocol Buffers**, traduciendo solicitudes JSON/REST a llamadas gRPC de baja latencia.
- **user-service (Spring Boot):** Encargado de la lógica de perfiles y seguridad en el puerto :9090.
- **inventario_service:** Gestiona la disponibilidad y el catálogo en el puerto :50053.
- **reservation_service:** Orquestador principal del proceso transaccional en el puerto :50052.
- **notification_service:** Gestiona el despacho de correos electrónicos transaccionales en el puerto :50051.
- **Persistencia (PostgreSQL):** Cada microservicio posee su propia base de datos aislada (puerto :5432 dentro de su contenedor) para garantizar el desacoplamiento.

Diagrama de Arquitectura



Flujo de Comunicación

Se implementa una estrategia híbrida de protocolos:

1. **Comunicación Externa:** Se utiliza REST/JSON para facilitar la integración con el navegador y herramientas estándar de depuración.
2. **Comunicación Interna:** Se emplea gRPC (HTTP/2). Esta elección es crítica para reducir el overhead de serialización y asegurar contratos estrictos entre servicios mediante Protobuf.

3. Análisis de Decisiones Técnicas y Trade-offs

Como Arquitecto Senior, he evaluado las siguientes decisiones clave basándose en el equilibrio entre agilidad de desarrollo y escalabilidad operativa:

3.1. Microservicios

- **Estrategia:** Descomposición funcional en dominios de negocio (Usuarios, Inventario, Reservas, Notificaciones).
- **Beneficios:** Independencia de despliegue y aislamiento de fallos, permitiendo que el inventario escale independientemente de las notificaciones, resolviendo así el problema de la alta concurrencia.
- **Compromisos:** Incremento en la complejidad operacional y latencia de red debido al salto entre servicios.
- **Alternativa Descartada:** Monolito, rechazado por generar un acoplamiento excesivo que impediría el escalado granular.

3.2. API Gateway (Patrón BFF)

- **Estrategia:** Punto de entrada único para la orquestación y seguridad.
- **Beneficios:** Centralización de CORS, autenticación y simplificación de la lógica del frontend al no exponer microservicios individuales.
- **Compromisos:** Introduce un salto de red adicional y un posible punto único de fallo (SPOF) que requiere estrategias de alta disponibilidad.
- **Alternativa Descartada:** Exposición directa de microservicios, descartada por riesgos de seguridad y complejidad excesiva en la gestión de endpoints desde el cliente.

3.3. gRPC para Comunicación Interna

- **Estrategia:** Uso de llamadas remotas binarias sobre HTTP/2.
- **Beneficios:** Alta eficiencia en la transmisión de datos y tipado fuerte.
- **Compromisos:** Curva de aprendizaje elevada y dificultad para realizar pruebas manuales sin herramientas especializadas (como `grpcurl`).
- **Alternativa Descartada:** REST/JSON interno, descartado por el alto costo computacional de la serialización en flujos de comunicación intensiva.

3.4. REST para Comunicación Externa

- **Estrategia:** Interfaz HTTP estándar para el consumo desde el Frontend.
- **Beneficios:** Universalidad, facilidad de depuración y compatibilidad nativa con Next.js sin necesidad de capas intermedias complejas.
- **Compromisos:** Mayor verbosidad en comparación con protocolos binarios.
- **Alternativa Descartada:** gRPC-Web, descartado por requerir proxies adicionales como Envoy, lo que complicaría innecesariamente la infraestructura actual.

3.5. Base de Datos por Servicio (Database per Service)

- **Estrategia:** Aislamiento físico de la persistencia para cada dominio.
- **Beneficios:** Desacoplamiento total de esquemas; la evolución de un servicio no impacta la integridad de otros.
- **Compromisos:** **Sacrificio de transacciones ACID distribuidas.** Se debe gestionar la **consistencia eventual** y diseñar flujos de compensación.
- **Alternativa Descartada:** Base de datos compartida, considerada un antipatrón que rompe la autonomía de los microservicios.

3.6. Docker Compose

- **Estrategia:** Orquestación de contenedores para el ciclo de vida del desarrollo.
- **Beneficios:** Agilidad extrema para levantar el ecosistema completo con un comando único.
- **Compromisos:** Falta de capacidades avanzadas de auto-escalado y gestión de salud (health checks) de nivel producción.
- **Alternativa Descartada:** Kubernetes (Minikube), descartado por la complejidad operativa desproporcionada para la fase actual del proyecto.

3.7. Poliglotismo

- **Estrategia:** Selección de stacks según idoneidad (Java para seguridad empresarial, FastAPI para asincronía, Go para rendimiento).
- **Beneficios:** Permite aprovechar las virtudes específicas de cada ecosistema (ej. Spring Boot Security).
- **Compromisos:** Mayor carga cognitiva para el equipo y pipelines de CI/CD más heterogéneos.

4. Catálogo de Microservicios y Comunicación gRPC

Servicio	Stack Tecnológico	Responsabilidad	Métodos / Flujos gRPC
api-gateway	Python (FastAPI)	BFF, ruteo y traducción de protocolos.	Client calls: <code>CreateUser</code> , <code>SearchAvailability</code> , <code>CreateReservation</code> .
user-service	Java (Spring Boot)	Gestión de identidad y persistencia de perfiles.	<code>CreateUser</code> , <code>Authenticate</code> .
inventario_servicio	Go / Python	Gestión de catálogo y estados de disponibilidad.	<code>GetAvailability</code> , <code>ValidateStock</code> , <code>UpdateInventory</code> .
reservation_servicio	Go / Python	Orquestador de transacciones de reserva.	<code>CreateReservation</code> , <code>GetReservationStatus</code> , <code>ListReservations</code> .
notification_servicio	Go / Python	Despacho de alertas y correos (vía Resend).	<code>SendEmailNotification</code> , <code>GetNotificationStatus</code> .

5. Casos de Uso y Flujos Técnicos Secuenciales

5.1. Registro de Usuario

- **Capa de Usuario:** El cliente envía datos de perfil; recibe un ID de confirmación.
- **Capa Técnica:**
 1. El Gateway recibe `POST /users` (REST).
 2. El Gateway actúa como cliente gRPC e invoca `CreateUser` en el user-service.
 3. El user-service procesa el hash de la contraseña y persiste en `users-db`.

5.2. Login / Autenticación

- **Capa de Usuario:** El cliente envía credenciales; recibe objeto de usuario validado.
- **Capa Técnica:**
 1. El Gateway recibe `POST /login`.
 2. Realiza llamada gRPC `Authenticate` al `user-service`.
 3. El servicio valida contra `users-db` y devuelve el estado de autenticación.

5.3. Búsqueda de Habitaciones

- **Capa de Usuario:** El cliente filtra por fecha y ubicación; recibe lista de opciones.
- **Capa Técnica:**
 1. El Gateway recibe `POST /api/inventory/search`.
 2. Llama al método gRPC `GetAvailability` en `inventario_service`.
 3. El servicio consulta `inventario_db` aplicando filtros de disponibilidad real.

5.4. Creación de Reserva (Orquestación Distribuida)

- **Capa de Usuario:** El cliente confirma la reserva; recibe ID de transacción y monto total.
- **Capa Técnica:**
 1. Petición `POST /reservations` al Gateway, mapeada internamente a `reservation_service`.
 2. **Orquestación:** El `reservation_service` ejecuta llamadas gRPC secuenciales:
 - Valida identidad llamando a `user-service`.
 - Verifica y bloquea stock llamando a `inventario_service` (`ValidateStock`).
 3. Persistencia de la reserva en estado "Confirmada" en `reservas_db`.
 4. **Notificación:** De forma sincrónica o asincrónica (según configuración de red), se invoca al `notification_service` para el envío del comprobante, registrando el evento en `notificaciones_db`.

6. Estado Operacional y Conclusiones

Estado Actual y Limitaciones

El sistema es funcional en sus flujos críticos de negocio, aunque presenta áreas de mejora técnica identificadas:

- **Funcionalidades Pendientes:** Ausencia de lógica para la cancelación de reservas y su correspondiente liberación de inventario.
- **Retos de Consistencia:** Al operar bajo un modelo de Consistencia Eventual, existe un riesgo técnico si el `reservation_service` confirma una transacción pero el `inventario_service` falla. Es imperativo evolucionar hacia la implementación de

un Patrón Saga para gestionar transacciones compensatorias (rollbacks distribuidos).

- **Infraestructura:** Se recomienda la transición hacia un Service Mesh (como Istio o Linkerd) para el descubrimiento dinámico de servicios y la observabilidad centralizada, eliminando el hardcoding de hosts.

Conclusión Técnica

Origen X constituye una arquitectura distribuida sólida y bien desacoplada. El valor del sistema reside en su capacidad para integrar tecnologías políglotas bajo contratos gRPC estrictos, garantizando eficiencia y escalabilidad. La adopción del patrón BFF y la persistencia aislada sitúa al proyecto en una posición ventajosa para una migración transparente hacia entornos cloud-native de alta disponibilidad.

Link Repositorio:

<https://github.com/darkkings19/Proyecto-SD-Reserva-de-Hoteles.git>

El readme se encuentra en el repositorio.